



**TED UNIVERSITY**  
**Faculty of Engineering**

**CMPE 491/SENG491**

**Senior Project I**

**LLM Valley**

**Project Specifications Report**

**TEAM 21**

**28.11.25**

**by**

Yusuf Sadi Pesen

Deniz Şeker

Egemen Asım Ersoy

Reyhan Sena Çimen

**Supervisor:**

Dr.Emin Kuğu

**Jury Members:**

Dr. Ulaş Güleç

Dr. Eren Ulu

|                                      |    |
|--------------------------------------|----|
| 1. Introduction.....                 | 2  |
| 2. Proposed system .....             | 3  |
| 2.1 Overview .....                   | 3  |
| 2.2 Functional Requirements .....    | 4  |
| 2.3 Nonfunctional Requirements ..... | 4  |
| 2.4 Pseudo requirements .....        | 5  |
| 2.5 System models .....              | 6  |
| 2.5.1 Scenarios .....                | 6  |
| 2.5.2 Use case model .....           | 8  |
| 2.5.3 Object and class model .....   | 10 |
| 2.5.4 Dynamic models .....           | 12 |
| 3. Glossary .....                    | 13 |
| 4. References .....                  | 14 |

# 1. Introduction

LLMs' rapid advancement has opened up new possibilities for interactive digital systems, such as their application in video gaming contexts. Conventional NPC dialogue systems have limited narrative depth and repetitious exchanges since they rely on prewritten scripts. More flexible and expressive NPC actions are becoming more and more necessary as player expectations for immersion and dynamic storytelling rise. By investigating how LLMs might be integrated into a small-scale life simulation game to produce organic, context-aware dialogues that raise player engagement, this study fills that demand.

While procedural content creation and adaptive difficulty have been the primary goals of current AI game applications, the use of LLMs for real-time dialogue is still mostly experimental. Current methods usually require extensive manual dialogue tree authoring or are not sustainable. By allowing NPCs to react intelligently to player input, update their emotional states, and engage in meaningful in-game interactions, this research explores if an LLM-driven system can provide a more agile and sustainable alternative.

LLM Valley, the game developed for this project, serves as a small proof-of-concept designed to assess the feasibility, stability, and player utility of LLM-powered NPCs. In the demo, players engage with four to five NPCs that can provide lively dialogue while performing basic life simulation tasks like farming, trading, and social interaction in a comfortable village setting. Alongside its primary gameplay loop, the project features stat systems (Love, Trust, Friendship, Mental), a day/night cycle, and a minimal trading and farming system to provide meaningful interaction scenarios.

This study aims to examine the planning, execution, and assessment of integrating LLMs into interactive NPC systems in a prototype playable game. As a result, the questions that follow are examined in this report:

1. How viable it is for an LLM-driven dialogue system to function quickly and reliably in a real-time gaming environment?
2. How effectively can dynamic dialogue support the immersion and the relationship-building in a community simulation setting?
3. What are the design and technical difficulties in integrating LLMs into gaming mechanics like trading interactions or stat progression?
4. How viable and scalable is this method for upcoming game development scenarios?

To answer these questions, the project prioritizes the following objectives:

1. Implement a functional LLM-based NPC interaction system using an applicable API.
2. Develop a simple but complete simulation loop that includes social interaction, trading, and farming.

3. Integrate player and NPC stat systems that are affected by dialogue.
4. Produce a stable Windows demo showcasing reliable gameplay and LLM behavior.

By looking at both the technical architecture and user experience of LLM Valley., this analysis aims to provide insights into the emerging field of AI-driven narrative interaction and contribute a foundation for future research and development in LLM-integrated games.

## **2. Proposed system**

### **2.1 Overview**

This project proposes the development of a narrative-driven life simulation demo named "LLM Valley" that integrates Large Language Model technology into interactive non-player character systems. Developed on the Unity game engine and powered by Deepseek or Gemini APIs, the system enables players to communicate freely and uniquely in natural language via keyboard, replacing pre-written text. The project aims to demonstrate the integration of AI-based text generation into real-time gameplay loops (farming, trading, relationship building) and make this accessible on mid-range PC hardware. The system is designed to offer an immersive experience with emotional depth using a modular structure requiring no database and keyword-based statistical analysis.

### **2.2 Functional Requirements**

#### **FR-1 LLM-Driven Dialogue System**

The application must accept free-text input from the player, transmit it to an LLM API (Deepseek or Gemini), and display the returned response as NPC dialogue within the game interface.

#### **FR-2 Keyword Based Stat Updates**

The system must analyze the player inputs and continuously update the NPC and the player common stats (Love, Trust, Friendship, Mental) based on specific keywords and phrases.

#### **FR-3 Farming Mechanics**

The player must be able to plant seeds on a designated grid, water crops, and harvest fully grown produce (such as wheat, tomatoes, and carrots).

#### **FR-4 Trading System**

The application must allow players to purchase seeds or gift items and sell their produced goods to earn in-game currency through a market interface.

#### **FR-5 Time & Cycle Management**

The game must advance the in-game day upon the player performing the "sleep" action and refresh/restore the player's "Mental" statistic.

#### **FR-6 Save/Load Functionality**

The system must save the current game stats (passed day count, inventory and equipments owned, emotional statistics) to a single json file and be able to restore the game from this file when the player came back.

#### **FR-7 User Interface (UI)**

The game must display chat history, player inventory, and current statistic bars (Love, Trust, Popularity, etc.) in a simple and understandable interface.

## **2.3 Nonfunctional Requirements**

#### **NFR-1 Performance**

**Response Latency:** Responses from the LLM API are targeted to be presented to the player within 5 seconds or less for 90% of inputs.

**Resource Efficiency:** The game must run smoothly on mid-range hardware (2018 and later, Windows 10+) without requiring additional hardware.

#### **NFR-2 Reliability**

**Stability:** The demo must run in computer without crashing for a full gameplay cycle (e.g. 7 days in-game format).

**Error Handling:** If the internet connection is lost or an API error occurs, the system must response with a default message like "I didn't understand that" without disrupting the flow for better experience.

#### **NFR-3 Ethical & Safety**

**Content Filtering:** LLM prompts must be filtered to prevent the generation of harmful, discriminatory, or offensive content, including protection against jailbreaks.

**Data Privacy:** Player data must not be stored on servers; only the current session context is processed temporarily.

#### **NFR-4 Usability**

**Intuitive Control:** The interface must be simple enough for players to use the system with a keyboard and mouse without needing a tutorial mode.

#### **NFR-5 Maintainability**

**Modular Architecture:** The LLM integration must be modular, allowing for future switches to different APIs or updates to prompts without breaking the code structure.

## **2.4 Pseudo requirements**

### **PR-1: Dialogue Tone**

NPC tone should change based on relationship stats (Love, Trust, Friendship, Mental) and NPC replies should stay varied and avoid repetition.

### **PR-2: Simple and Clear Language**

All UI messages and NPC dialogue should use easy, B2-level English. This helps non-native players understand the content without confusion.

### **PR-3: Soft Visual Style**

The game should follow a cozy color palette. While this is not required for gameplay, it improves the atmosphere and makes the world feel more relaxing.

### **PR-4: Simple and Clear UI**

The user interface should show only the most important information. Too many icons or numbers should be avoided to keep the screen calm and readable.

### **PR-5: Optional Accessibility Settings**

Players may be allowed to adjust text size, contrast theme, or chat fade-out speed. These options are not required but can improve comfort for long play sessions.

### **PR-6: NPC Emotion Changes**

Relationship stats should influence NPC tone gradually. Sudden or extreme mood shifts should be avoided to keep interactions natural.

### **PR-7: Soft Feedback for Player Actions**

When farming, trading, or talking, the game may give small hints like a short sound, color blink, or message. This feedback is optional but helps players understand what happened. If LLM replies are slow, shows a “Thinking...” message.

### **PR-8: Future Friendly Structure**

Even though the current game is small, prompt and data files should be arranged in a way that makes future expansion easier (new NPCs, new crops, new areas).

## 2.5 System models

### 2.5.1 Scenarios

#### Scenario 1: First-Time Use Talking to an NPC

##### Preconditions:

- The player has installed and opened the game.
- Player has created the first character.
- LLM API connection is available.

##### Steps:

1. Player enters the village for the first time.
2. A short tutorial message appears: “Go talk to any villager to start your first conversation.”
3. Player approaches an NPC and presses the interaction key.
4. The dialogue window opens and the game gives an instruction: “Type your message below.”
5. Player writes a simple input: “Hello.”
6. The system sends the message to the LLM API.
7. The API returns a generated reply.
8. The game displays the NPC response on screen with the default tone and stats.
9. The stat system analyzes the player’s message (“hello”) and adds a small increase to Friendship.
10. Player continues the conversation or leaves.

##### Postconditions:

- First NPC conversation completed.
- Initial stat changes saved.
- The system confirms the dialogue module works normally.

#### Scenario 2: LLM API Error / Timeout Handling

##### Preconditions:

- Player is in a conversation with an NPC.
- LLM API becomes slow or temporarily unavailable.

##### Steps:

1. Player sends a message: “Can you tell me how your day was?”
2. The system transmits the request to the API.

3. After a few seconds, no response is returned (timeout or network error).
4. The game shows a temporary message: “Thinking...”
5. The error handler activates and replaces the missing response with a fallback line: “Sorry, I couldn’t think clearly for a moment. Can you ask again?”
6. The player message is stored so they can repeat it quickly.
7. Player resends or writes a new message.
8. Once the API connection is restored, normal dialogue continues.

#### **Postconditions:**

- Game continues without crashing.
- Error logged locally for debugging.
- No progress or stats are lost.

### **Scenario 3: Trading/Selling Interaction**

#### **Preconditions**

- The player has harvested items in inventory.
- The shopkeeper NPC is available in the market area.
- The LLM API is accessible for dynamic price calculation.
- The player has enough in-game currency to buy, if needed.

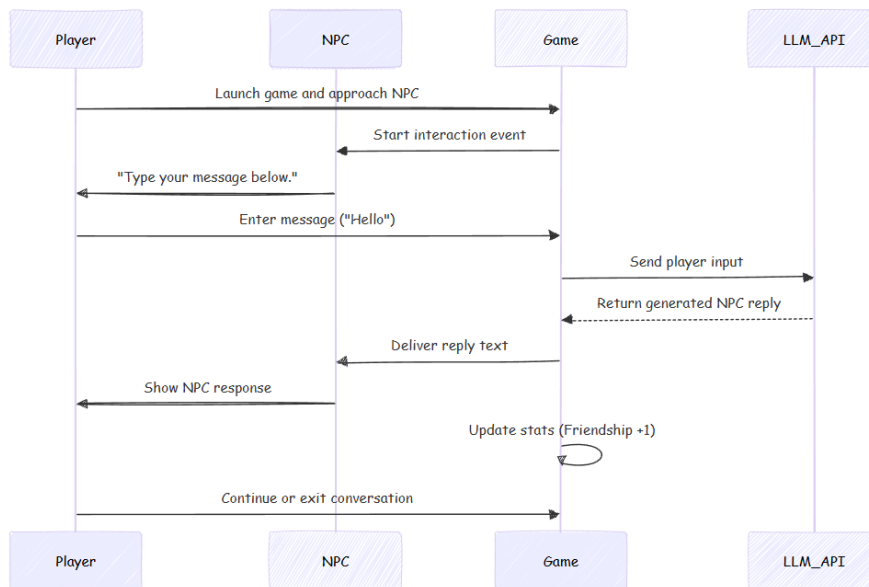
#### **Steps**

- The player walks to the market area and approaches the shopkeeper NPC.
- The player presses the interaction key to open the trading interface.
- The game displays two tabs: Buy and Sell.
- The player selects the Sell tab.
- The system shows all sellable items in the player's inventory.
- The player chooses an item and presses the Sell button.
- The game sends item data and the current friendship value to the LLM API.
- The LLM calculates an adjusted price (e.g., higher price if friendship is high, lower if it is low).
- The game displays the final price to the player.
- The player confirms the sale.
- The system removes the selected items from the inventory.
- The player’s in-game currency increases according to the LLM-generated price.
- The system shows a confirmation message
- The player may continue selling or switch to the Buy tab to purchase seeds or gifts.
- The player exits the trading interface and returns to normal gameplay.

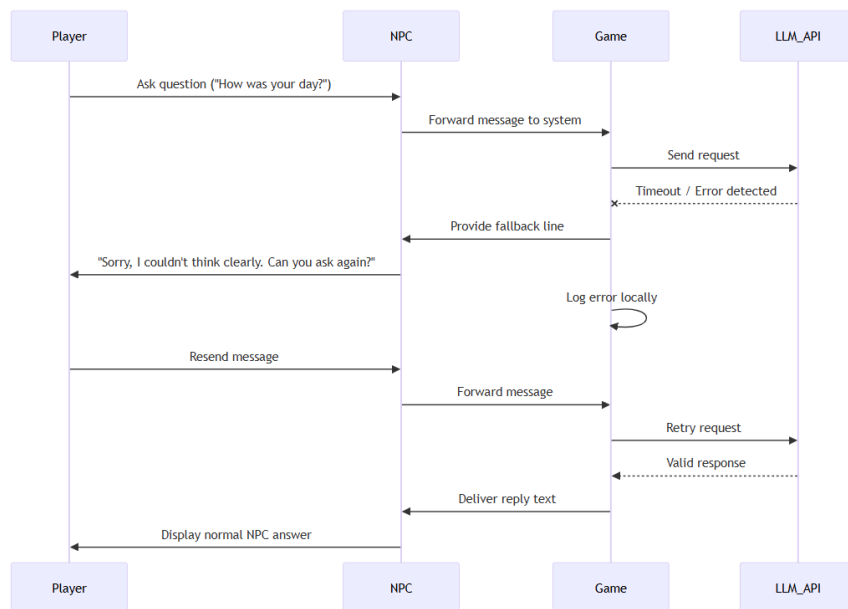
#### **Postconditions**

- Sold items are removed from the inventory.
- Coins are updated based on the LLM-adjusted value.
- The new state is saved for future sessions.
- The trading system confirms that the transaction was successful.

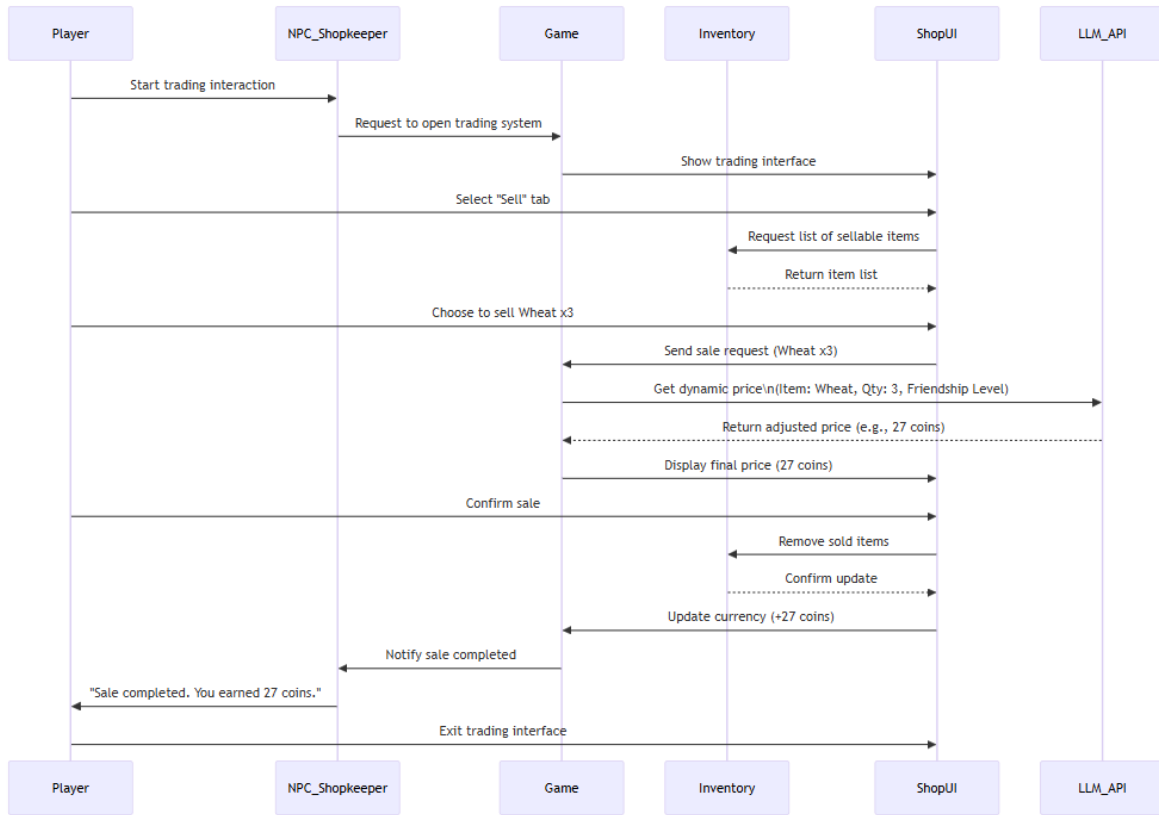
## 2.5.2 Use case model



**Figure 1: Scenario 1 – First-Time Use (NPC Dialogue) Explanation**

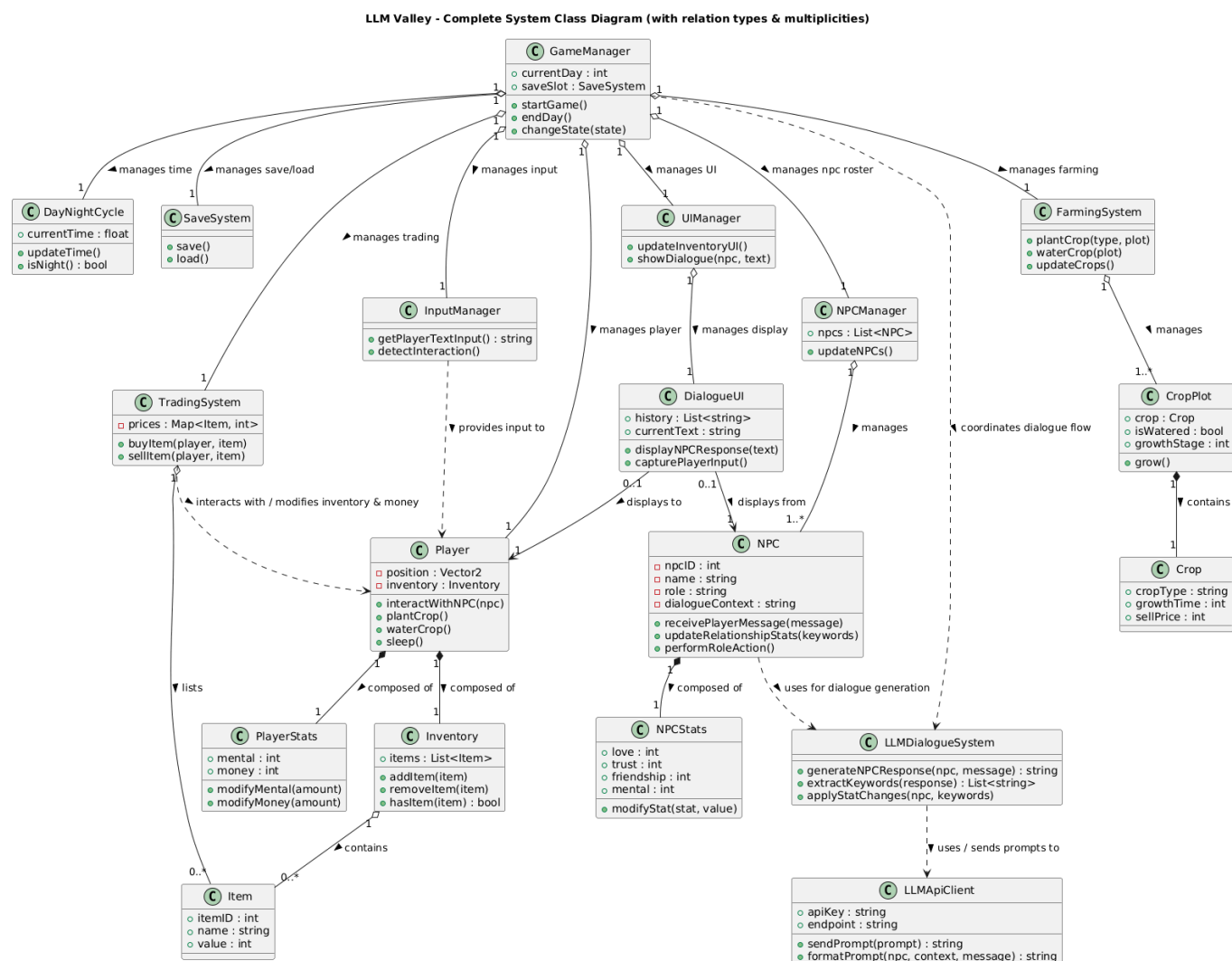


**Figure 2: Scenario 2 – LLM API Error / Timeout Handling Explanation**



**Figure 3: Scenario 3 – Trading / Selling Interaction Explanation**

### 2.5.3 Object and class model



**Figure 4: Uml graph for LLM Valley**

## 2.5.4 Dynamic models

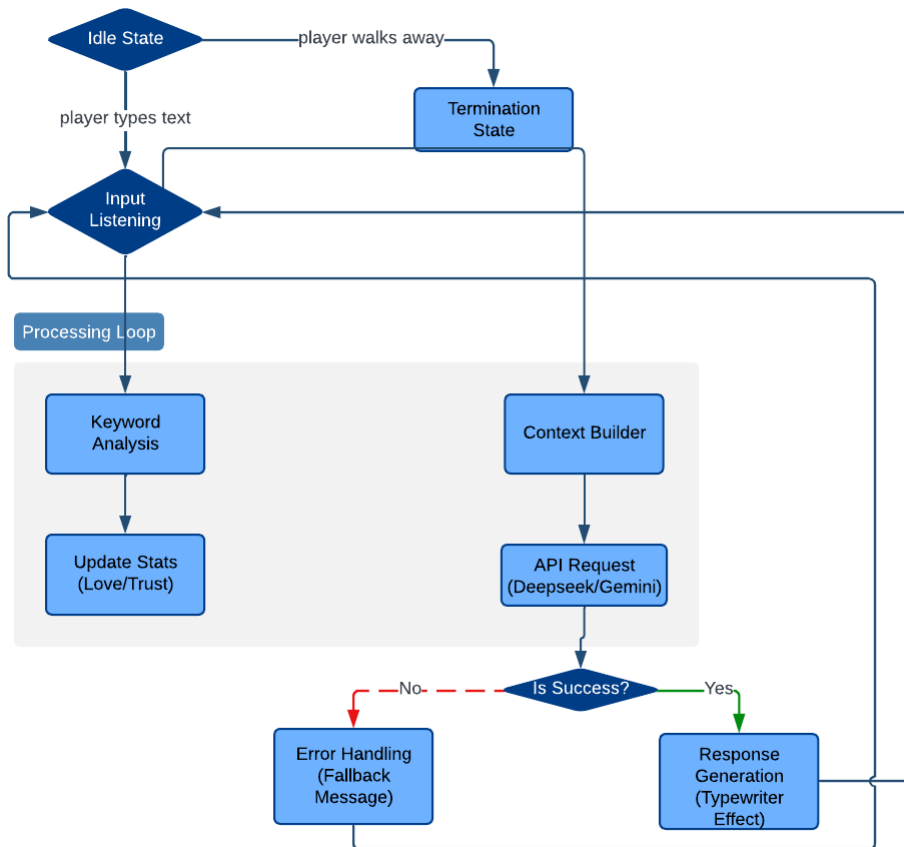
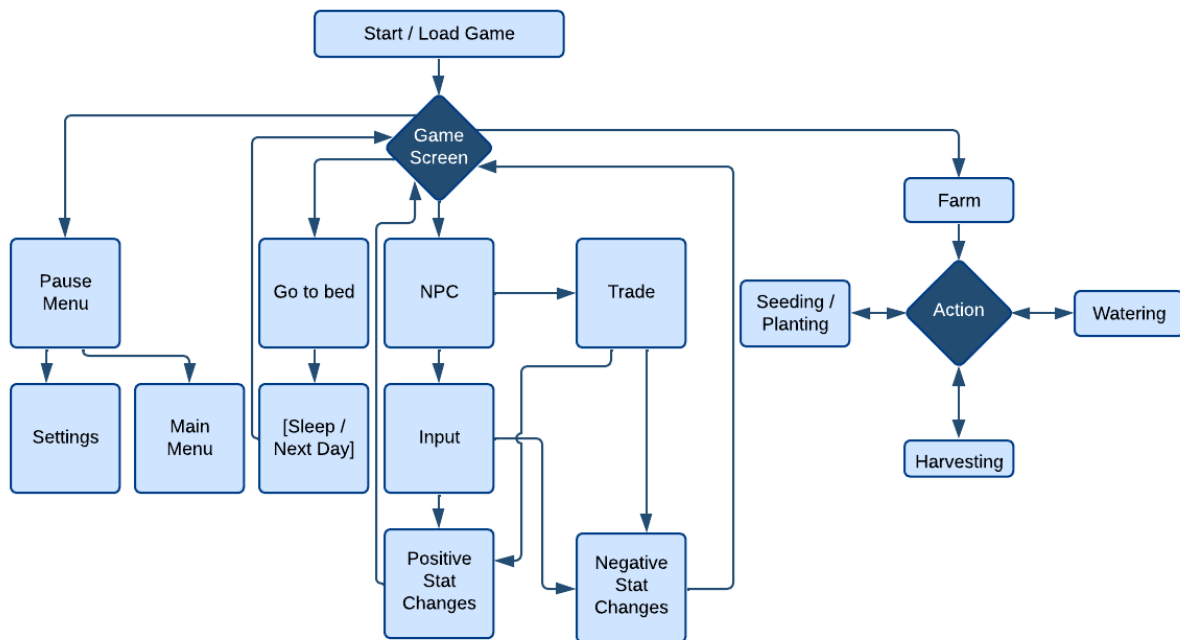


Figure 5: Uml graph for LLM Valley

## 2.5.5 Navigational Paths



**Figure 6: User interface - navigational paths and screen mock-ups**

### 3. Glossary

**NPC:** Non Player Character

**Gameplay Loop:** A repeating cycle of actions players perform during normal play (e.g., farm → trade → talk → sleep).

**In-Game Day / Day Cycle:** A simulated time cycle where events reset or progress after a "sleep" action.

**Stat System / Relationship Stats:** Variables representing emotional or social parameters (Love, Trust, Friendship, Mental).

**Inventory System:** A structure that holds items the player owns.

**Tokenizer:** A preprocessing tool that splits text into pieces (tokens) used by the LLM.

**Fallback Response:** A default message given when the LLM API fails or times out.

**Script / MonoBehaviour:** A C# class attached to Unity GameObjects to control behavior.

**Prefab:** A reusable pre-configured object (NPC, crop plot, item, etc.) that can be instantiated in the game.

**GameObject:** The basic building block of Unity scenes that can hold components such as sprites, colliders, and scripts.

**Scene:** A collection of objects forming a game level.

**UI Canvas:** Unity's system for rendering user interface elements.

## 4. References

- [1] L. Song, "LLM-Driven NPCs: Cross-Platform Dialogue System for Games and Social Platforms," 2025.
- [2] L. M. Csepregi, "The Effect of Context-aware LLM-based NPC Conversations on Player Engagement in Role-playing Video Games," Department of Architecture, Design and Media Technology, Aalborg University, Copenhagen, Denmark, 2024.
- [3] G. Zeng, "A review of AI-based game NPCs research," *Applied and Computational Engineering*, vol. 15, pp. 155–159, 2023.
- [4] Unity Technologies, "Unity Manual and Scripting API," *Unity Documentation*, 2024. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [5] Unity Technologies, "Using JSONUtility for Saving and Loading Data," *Unity Documentation*, 2023. [Online]. Available: <https://docs.unity3d.com/ScriptReference/JsonUtility.html>
- [6] ConcernedApe. Stardew Valley [Video game]. ConcernedApe, 2016. Available at: [https://store.steampowered.com/app/413150/Stardew\\_Valley/](https://store.steampowered.com/app/413150/Stardew_Valley/)
- [7] Sam Edwards. Travellers Rest [Video game]. Isolated Games, 2020. Available at: [https://store.steampowered.com/app/1139980/Travellers\\_Rest/](https://store.steampowered.com/app/1139980/Travellers_Rest/)
- [8] Fair Games Studio. Necesse [Video game]. Fair Games Studio, 2019. Available at: <https://store.steampowered.com/app/1169040/Necesse/>
- [9] GameDeveloper.com, "Design Patterns of Farming and Life Simulation Games," 2020. [Online]. Available: <https://www.gamedeveloper.com/design>
- [10] *Object-Oriented Software Engineering, Using UML, Patterns, and Java, 2nd Edition*, by Bernd Bruegge and Allen H. Dutoit, Prentice-Hall, 2004, ISBN: 0-13-047110-0.